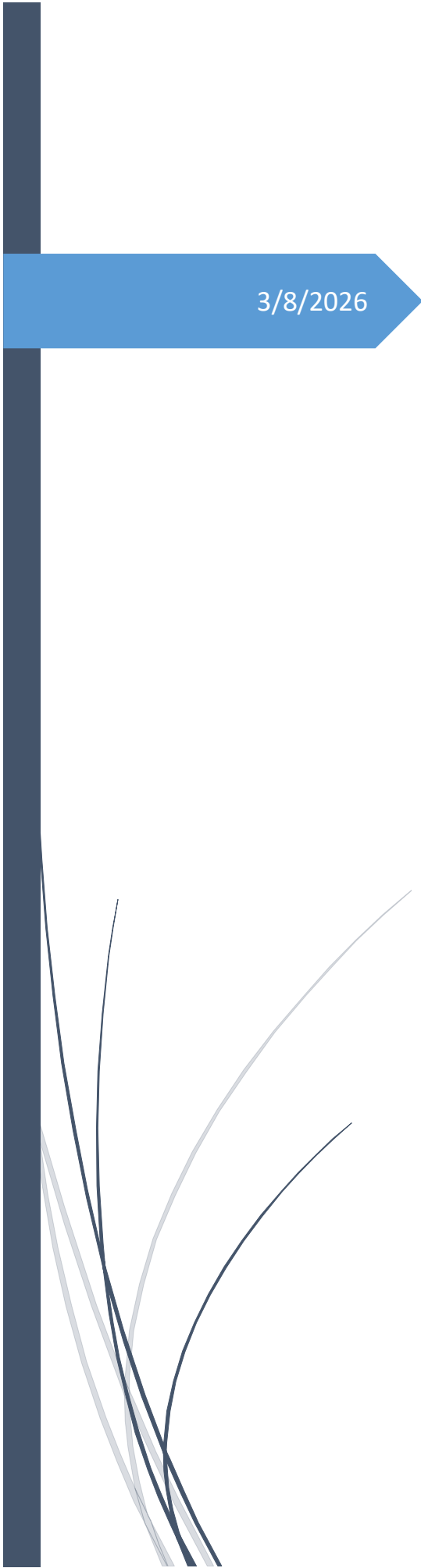


A dark blue vertical bar runs down the left side of the page. A blue arrow-shaped banner points to the right from this bar, containing the date. Below the banner, several thin, curved lines in dark blue and light grey sweep upwards from the bottom left corner.

3/8/2026

Project Proposal

Submitted to: Mr. Syed Ahmed Khan

Project Title:

Carefree – Aiding Caretakers

Team Members:

- Simal Hassan (24k-0688)
 - Laiba Jamil (24k-0812)
 - Ahmed Raza (24k-1010)
-

System Overview:

Carefree – Aiding Caretakers is a mobile-based application designed for Android devices to streamline and automate complex caretaking activities. The system is built using the Flutter framework for the frontend, connecting to a Java Spring Boot and Python backend through a three-tier Service Oriented Architecture (SOA).

The application is primarily used by two groups:

1. **Caretakers:** The primary users who manage patient records, vitals, medications, and tasks.
2. **Guardians:** The patient's emergency contacts who use the system to monitor the patient's condition and communicate with caretakers.

The overall purpose is to provide a comprehensive digital tool for in-home caregivers to manage health tracking, dietary planning, and financial recording for their patients.

Problem Statement:

Traditional in-home caretaking often involves tedious, manual, and complex management of multiple patient needs—ranging from medication schedules to dietary restrictions and financial tracking. This manual process is prone to errors, such as missing medication doses or failing to spot dangerous trends in vital signs.

This problem affects both professional and family caregivers, as well as the patients who depend on timely and accurate care. Automating these tasks is crucial to ensure patient safety, reduce the cognitive load on caretakers, and provide guardians with transparency regarding the patient's well-being.

Project Objectives:

- **Streamline Patient Management:** Automate the recording and updating of patient personal and medical details.
 - **Real-time Health Monitoring:** Enable daily tracking of vitals and symptoms with automated trend analysis.
 - **Intelligent Medication Scheduling:** Provide a robust system for scheduling, tracking, and alerting for patient medications.
 - **AI-Powered Dietary Assistance:** Utilize AI to flag meals that violate dietary restrictions or fail to meet nutritional goals.
 - **Financial Accountability:** Track all patient-related expenses and generate comprehensive financial reports.
 - **Predictive Cost Analysis:** Implement ML algorithms to estimate future medical costs based on historical patient data.
 - **Enhanced Communication:** Facilitate secure and classified messaging between caretakers and guardians.
 - **Automated Alerting:** Trigger critical notifications for abnormal vitals, upcoming appointments, and low medication stock.
-

Software Design Specification (SDS) Outline:

- **Use Case Diagram**

The Use Case Diagram provides a high-level view of system functionality from the perspective of its users. It illustrates how the two primary actors — Caretaker and Guardian — interact with the twelve core services offered by the Carefree system.

The Caretaker actor is associated with use cases covering patient management, health tracking, medication management, task assignment, dietary planning, and financial recording. The Guardian actor interacts with use cases related to patient monitoring, alert reception, health report viewing, and caretaker communication. Both actors share the Login use case as a common entry point.

□ Use Case Diagram

Actors: Caretaker, Guardian | Use Cases: Login, Manage Patient Records, Record Vitals, Manage Medications, Assign Tasks, Plan Diet, Record Expenses, AI Meal Flagger, View Patient Status, Receive Alerts, View Reports, Message Caretaker

• Class Diagram

The Class Diagram represents the static structure of the Carefree system, defining all major entities, their attributes, methods, and the relationships between them. The design follows object-oriented principles including inheritance, encapsulation, and association.

The Person class serves as the abstract base class, from which Caretaker and Guardian are derived through inheritance. The Patient class maintains composition relationships with VitalsRecord and DietPlan, and association relationships with Medication, Task, and Expense. This structure ensures that patient-specific records are tightly coupled to their owner while allowing shared references where appropriate.

▣ Class Diagram

Classes: Person (abstract base), Caretaker, Guardian, Patient, VitalsRecord, Medication, Task, DietPlan, Expense | Relationships: Inheritance, Composition, Association, Dependency

• Activity Diagram

The Activity Diagram models the flow of actions within a specific system process. For Carefree, the diagram focuses on the Patient Information Management workflow, which represents one of the most frequently performed operations by a Caretaker.

The flow begins when the Caretaker accesses the Dashboard and selects the option to add a new patient. The system then presents the patient intake form. Upon submission, the input is validated — if validation fails, the user is prompted to

correct the data; if successful, the record is saved to the database and a confirmation is displayed. Decision nodes and swim lanes are used to distinguish system and user responsibilities throughout the process.

Activity Diagram — Patient Information Management

Flow: Dashboard → Select Add Patient → Fill Form → Validate Input → [Valid] Save to Database → Confirmation | [Invalid] → Show Errors → Re-enter Data

• Sequence Diagram

The Sequence Diagram captures the dynamic interaction between system participants over time, showing the order in which messages are exchanged. The selected scenario illustrates the process of logging patient vitals, which involves coordination between the Caretaker, the application UI, the backend Analysis Engine, and the Database.

The Caretaker initiates the interaction by entering vitals data through the UI. The UI forwards the data to the Analysis Engine, which validates and processes the readings. If an anomaly is detected — for example, abnormally high blood pressure — an alert is generated and dispatched to the Guardian. The processed record is then persisted to the Database, and a success response is returned to the UI for display.

Sequence Diagram — Log Patient Vitals

Participants: Caretaker, Application UI, Analysis Engine (Python), Database | Flow: Enter Vitals → Validate → Analyze → [Anomaly Detected] Alert Guardian → Save Record → Confirm

- **Collaboration Diagram**

The Collaboration Diagram, also referred to as a Communication Diagram in UML 2.x, complements the Sequence Diagram by emphasizing the structural relationships between objects rather than the chronological order of messages. For Carefree, this diagram is applied to the Guardian Communication service.

The diagram illustrates the object links between the Guardian, the NotificationController, the AlertService, and the Patient entity. Numbered messages indicate the sequence of communication, showing how an alert originates from a vitals anomaly and propagates through the system to reach the Guardian's device. This view is particularly useful for understanding the connectivity of objects involved in real-time notification handling.

□ **Collaboration Diagram — Guardian Communication Service**

Objects: Guardian, NotificationController, AlertService, Patient | Links: Numbered messages showing alert propagation from vitals anomaly to Guardian notification

Object-Oriented Design Considerations:

The system follows strict Object-Oriented principles to ensure maintainability:

- **Encapsulation:** Sensitive patient data (e.g., diagnosis, vitals) and guardian contact details are encapsulated within the **Patient** and **Person** classes, with access controlled through getter and setter methods.
- **Inheritance:** A base **Person** class contains common attributes like **Name** and **Contact**. The **Guardian** and **Caretaker** classes inherit from this base class to avoid redundancy.
- **Polymorphism:** The system uses polymorphic reporting services where different types of reports (Vitals, Finance, Symptoms) are generated through a unified **generate_reports()** interface.
- **Abstraction:** Complexity in ML processing and AI meal detection is abstracted away from the user; the caretaker only interacts with high-level functions like **Predict_medical_cost()** or **Upload image**.

Expected System Modules:

- **User Management Module:** Handles sign-up, sign-in, and role-based access for caretakers and guardians.
 - **Patient Record Service:** Manages CRUD operations for patient profiles, medical history, and document storage.
 - **Health Tracking Service:** Logs daily vitals and symptoms, performing trend analysis and generating alerts.
 - **Medication & Appointment Module:** Schedules healthcare events and automates medication stock tracking.
 - **Task Management Module:** Organizes caretaker duties with priority-based alerts.
 - **Dietary Management Module:** Includes meal template building and AI-driven meal flagging.
 - **Financial Tracking Module:** Logs medical expenses and generates downloadable PDF/CSV reports.
 - **Communication & Notification Service:** Provides secure in-app messaging and custom push notifications.
-

Future Extensions:

- **Direct Fund Transfer:** Integration of a payment gateway to allow guardians to pay caretaker wages directly through the app.
- **Advanced ML Models:** Scaling the cost predictor to include multi-variate analysis of long-term health trends.
- **Wearable Integration:** Proposed: Syncing vitals automatically from smart watches or health bands to replace manual entry.

THE END